

Stage 5: Production Modular Monolith Architecture Specification

Alight International Multi-Vendor Marketplace Document Version: 5.0.0 (Production-Ready) Runtime Platform: Java 21 LTS | Spring Boot 3.3.4 | Spring Data JPA | Spring Security 6 Architecture Pattern: Modular Monolith with Asynchronous In-Memory Domain Event Bus

1. Architectural Principles & Encapsulation Rules

- 1. Strict Module Boundary Isolation:** Each domain module resides in `com.alight.marketplace.modules.<module_name>` containing its own private controllers, entities, repositories, and services.
- 2. Loosely-Coupled Inter-Module Communication:**
 - **Synchronous Direct Calls:** Used strictly for read operations or immediate transaction composition via public Service interfaces (e.g. `RbacService`, `UserService`, `ResourceSecurityService`).
 - **Asynchronous Domain Events:** Used for cross-module side-effects (e.g. notifying vendors on order creation, creating audit logs, sending push/email notifications, unlocking escrow) published via Spring's `ApplicationEventPublisher`.
- 3. MDC Correlation Propagation:** All background `@Async` threads automatically receive the caller's request `traceId` via `MdcTaskDecorator` to ensure end-to-end distributed tracing across async operations.
- 4. Deterministic Concurrency Control:** Entity modifications on mutable transactional state use JPA `@Version` optimistic locking, handled globally with `409 Conflict` (`OPTIMISTIC_LOCK_CONFLICT`).

2. 28-Module Architecture Catalog

Module	Package Root	Primary Domain Responsibility	Key Events Published / Consumed
auth	modules.auth	JWT issuance, refresh token rotation, authentication entry points	Consumes: Login events
user	modules.user	User profiles, shipping addresses, RBAC roles & permissions	Consumes: User creation
vendor	modules.vendor	Store profiles, KYC validation, bank accounts, pickup addresses	Publishes: VendorStatusChangedEvent
product	modules.product	Hardware catalog, variants, attributes, images, approval state	Consumes: Stock changes
category	modules.category	Hierarchical category taxonomy, brand definitions	-
inventory	modules.inventory	Multi-warehouse stock, temporary reservations (15m TTL), logs	Publishes: StockReservedEvent
cart	modules.cart	Multi-vendor user & guest carts, currency line items	-
order	modules.order	Master order splitting (#10001 → #10001-A , #10001-B), fulfillment	Publishes: OrderCreatedEvent , OrderStatusUpdatedEvent
checkout	modules.checkout	Multi-step checkout pipeline, address validation, tax calculation	Publishes: Checkout triggers
payment	modules.payment	Gateway integrations (Razorpay/Stripe), webhook idempotency	Publishes: PaymentSucceededEvent
settlement	modules.settlement	Vendor wallets, escrow hold periods	Consumes: PaymentSucceededEvent , OrderStatusUpdatedEvent

Module	Package Root	Primary Domain Responsibility	Key Events Published / Consumed
		(7d), payouts, commission invoices	
logistics	modules.logistics	Carrier abstraction (Delhivery, BlueDart), rate matrix, manifests	Consumes: <code>OrderStatusUpdatedEvent</code>
returns	modules.returns	7-day RMA claims, photo proof upload, vendor action, reverse logistics	Publishes: <code>RmaSubmittedEvent</code>
coupon	modules.coupon	Global & vendor discount coupons, redemption limits, scopes	-
pricing	modules.pricing	B2B bulk tier pricing matrix, flash sale quotas	-
currency	modules.currency	Multi-currency FX exchange rates (INR, USD, AED, EUR)	-
tax	modules.tax	GST HSN classifications (18% Hardware, 12% Brass), IGST/CGST/SGST	-
review	modules.review	Verified customer reviews, rating calculations, helpful votes	-
support	modules.support	Threaded customer & vendor support tickets, dispute escalations	-
notification	modules.notification	Email, SMS, and in-app notification template dispatches	Consumes: All domain events
search	modules.search	PostgreSQL Trigram GIN faceted search & instant autocomplete	-
bundle	modules.bundle	Curated hardware sets (e.g. "Full Modular Kitchen Package")	-

Module	Package Root	Primary Domain Responsibility	Key Events Published / Consumed
quote	modules.quote	B2B Request For Quote (RFQ) Bill of Quantities engine	-
wishlist	modules.wishlist	Saved architectural moodboards and wishlist collections	-
personalization	modules.personalization	Recently viewed items, recommendations	-
analytics	modules.analytics	Vendor dashboards, platform executive metrics, GMV reports	-
audit	modules.audit	Immutable operational audit logs	Consumes: All security & admin events
health	modules.health	Live PostgreSQL and memory diagnostic health check	-

3. High-Level Modular Monolith Diagram (Mermaid)

MERMAID

```
graph TD
    subgraph ClientLayer [Client & Gateway Layer]
        CustomerStorefront["Customer Storefront (Next.js)"]
        VendorPortal["Vendor Portal (Next.js)"]
        AdminDashboard["Admin Operations Portal (Next.js)"]
    end

    subgraph SecurityFilter [Security & Traceability Layer]
        MDCFilter["MdcCorrelationFilter (traceId)"]
        JWTFilter["JwtAuthenticationFilter (Stateless RBAC)"]
        SecService["ResourceSecurityService (Tenant Isolation)"]
    end

    subgraph CoreModules [Modular Monolith Domain Engines]
        AuthMod["Auth & User Module"]
        CatalogMod["Product & Category Module"]
        InventoryMod["Multi-Warehouse Inventory"]
        OrderMod["Order & Checkout Pipeline"]
        PaymentMod["Payment & Webhook Engine"]
        SettlementMod["Vendor Escrow & Settlement"]
        LogisticsMod["Logistics & Courier Routing"]
        RmaMod["Returns & RMA Module"]
    end

    subgraph EventBusLayer [Asynchronous Event Bus]
        EventBus["Spring ApplicationEventPublisher (Async TaskExecutor)"]
        AuditSub["Audit Logging Subscriber"]
        NotifSub["Notification Subscriber (Email/SMS)"]
    end

    subgraph DatabaseLayer [PostgreSQL 18 Multi-Tenant Schema]
        PGDB["alight_db (71 Base Tables + Triggers + GIN Indexes)"]
    end

    ClientLayer --> MDCFilter --> JWTFilter --> SecService
    SecService --> CoreModules
    CoreModules --> EventBus
    EventBus --> AuditSub
    EventBus --> NotifSub
    CoreModules --> PGDB
```

4. Multi-Vendor Order & Event Lifecycle

MERMAID

```
sequenceDiagram
    autonumber
    actor Customer
    participant Checkout as CheckoutController
    participant Inventory as ProductInventoryController
    participant Order as OrderService
    participant EventBus as ApplicationEventPublisher
    participant DomainListener as DomainEventListener
    participant Settlement as SettlementService

    Customer->>Checkout: POST /api/v1/checkout/process
    Checkout->>Inventory: Reserve stock across warehouses (TTL 15m)
    Checkout->>Order: Create Master Order #10001
    Order->>Order: Split into Vendor Orders (#10001-A, #10001-B)
    Order->>EventBus: publishEvent(OrderCreatedEvent)
    EventBus-->>DomainListener: handleOrderCreated() [Async with MDC traceId]

    Customer->>Checkout: POST /api/v1/payments/verify
    Checkout->>EventBus: publishEvent(PaymentSucceededEvent)
    EventBus-->>Settlement: Credit Pending Escrow to Vendor Wallets
    EventBus-->>DomainListener: Dispatch Customer & Vendor Email Confirmations
```

5. Asynchronous Worker Pool Specifications (AsyncConfig)

JAVA

```
@Configuration
@EnableAsync
@EnableScheduling
public class AsyncConfig {
    // Core Pool: 5 worker threads
    // Max Pool: 25 worker threads
    // Queue: 100 queued events
    // Decorator: MdcTaskDecorator (preserves traceId context)
}
```